

Final Exam — Advanced Data Systems

Student Submission Template

Section: 2

Date Submitted: 12 December 2025

Part 0 — Data Modeling & Ingest (15 marks)

1. Provide the dataset URLs used (OWID CSV, TLC Parquet).

OWID COVID-19 dataset (CSV):

<https://raw.githubusercontent.com/owid/covid-19-data/master/public/data/owid-covid-data.csv>

NYC TLC Taxi Trips (Parquet – September 2024):

https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2024-09.parquet

2. Describe your import commands or process (mongoimport, Python script, etc.).

OWID (Reduced Column Import)

Imported using a custom Python script:

- Loaded CSV with Pandas
- Kept only 9 required fields to fit Atlas free tier
- Converted date → ISODate
- Replaced NaN with None
- Inserted documents using PyMongo insert_many

Script used:

```
df = pd.read_csv("owid-covid-data.csv", low_memory=False)
```

```
df = df[[
```

```
    "location", "continent", "date", "new_cases", "new_deaths",
```

```
    "people_fully_vaccinated_per_hundred", "hospital_beds_per_thousand",
```

```
    "stringency_index", "population"
```

```
]]
```

```
df["date"] = pd.to_datetime(df["date"])
```

```
df = df.where(pd.notnull(df), None)
```

```
records = df.to_dict(orient="records")
```

```
db.owid_daily.insert_many(records)
```

NYC TLC Taxi Trips (Sampled Import)

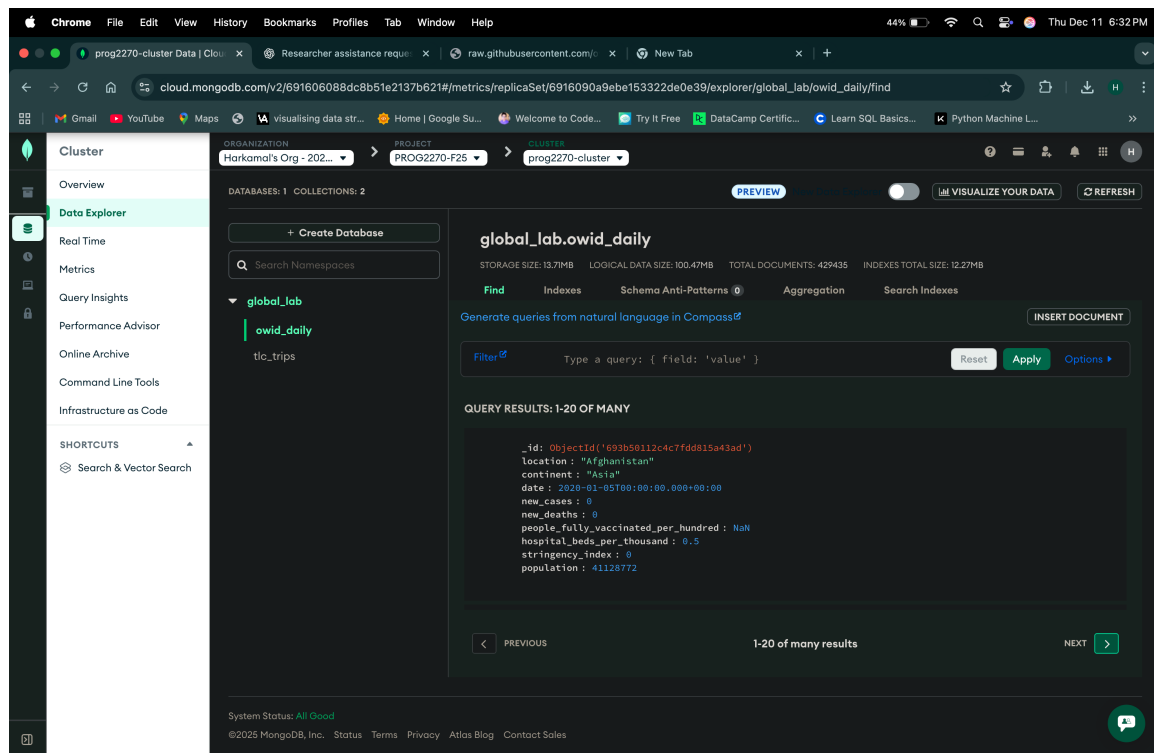
Imported using Python + Pandas + PyArrow:

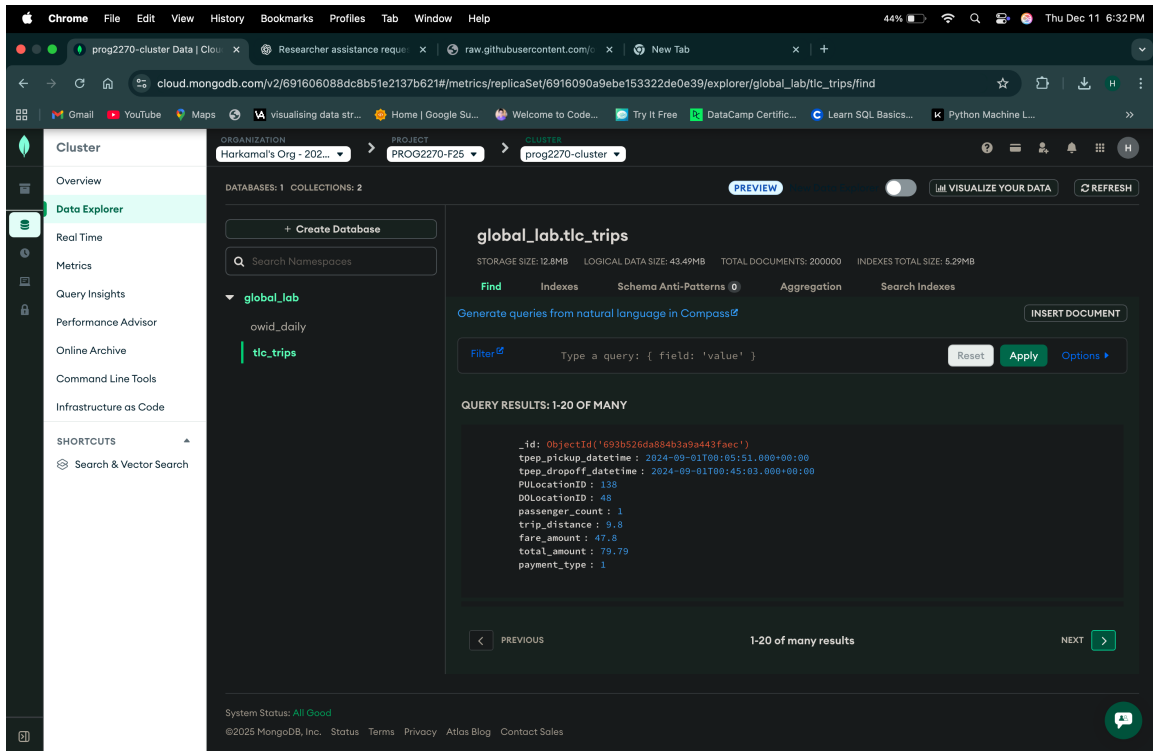
- Loaded Parquet file with Pandas
- Selected 9 required columns
- **Used first 200,000 rows** to stay within Atlas free-tier storage quota
- Inserted using PyMongo insert_many

Script used:

```
df = pd.read_parquet("yellow_tripdata_2024-09.parquet")
df = df[[
    "tpep_pickup_datetime", "tpep_dropoff_datetime", "PULocationID",
    "DOLocationID", "passenger_count", "trip_distance",
    "fare_amount", "total_amount", "payment_type"
]]
df = df.head(200000)
df["tpep_pickup_datetime"] = pd.to_datetime(df["tpep_pickup_datetime"])
df["tpep_dropoff_datetime"] = pd.to_datetime(df["tpep_dropoff_datetime"])
df = df.where(pd.notnull(df), None)
records = df.to_dict(orient="records")
db.tlc_trips.insert_many(records)
```

3. Include a screenshot of MongoDB Atlas collections (owid_daily, tlc_trips).





4. List and justify your index plan (1–2 sentences each).

OWID Collection Indexes

1. **{ location: 1, date: 1 }**
Used for all rolling window operations and filtering by country and date ranges; accelerates T1 and T2 pipelines.
2. **{ continent: 1, date: 1 }**
Supports grouping and filtering for continent-wide statistics; useful for analytical queries and dashboards.

TLC Collection Indexes

1. **{ PULocationID: 1, tpep_pickup_datetime: 1 }**
Required for hour-of-day aggregations, demand analysis, and fast lookup of trips per zone.
2. **{ tpep_pickup_datetime: 1 }**
Improves performance for time-series queries, moving averages, and percentile calculations in T5.

Task 1 — Pandemic Surge Detection (18 marks)

This task analyzes weekly COVID-19 surges by computing rolling averages over daily case counts. The workflow includes computing 7-day rolling means, 14-day lagged means, surge multipliers, ISO week grouping, and selecting the top 5 surge countries each week. Performance optimization was evaluated using MongoDB's `explain()` before and after indexing.

The pipeline performs the following operations:

1. Sorts all documents by `location` and `date`
2. Computes a 7-day rolling average of new cases using `$setWindowFields`
3. Computes a 14-day lagged average using `$shift`
4. Calculates the surge multiplier ($\text{avg_7d} / \text{avg_7d_lag14}$)
5. Extracts ISO year and ISO week values
6. Groups results by (year, week)
7. Selects the top 5 countries with highest surge multipliers
8. Sorts the weekly output chronologically

```
db.owid_daily.aggregate([
```

```
// Stage 1 – Sort data for window calculations
```

```
{
  $sort: { location: 1, date: 1 }
},
```

```
// Stage 2 – Compute 7-day rolling average of new cases
```

```
{
  $setWindowFields: {
    partitionBy: "$location",
    sortBy: { date: 1 },
    output: {
      avg_7d: {
```

```
    $avg: "$new_cases",  
    window: {  
      range: [-6, 0],  
      unit: "day"  
    }  
  }  
}  
}  
},  

```

// Stage 3 – Get the 7-day average from 14 days earlier

```
{  
  $setWindowFields: {  
    partitionBy: "$location",  
    sortBy: { date: 1 },  
    output: {  
      avg_7d_lag14: {  
        $shift: {  
          output: "$avg_7d",  
          by: -14  
        }  
      }  
    }  
  }  
},  

```

```
// Stage 4 – Compute surge multiplier
```

```
{  
  $addFields: {  
    surge_multiplier: {  
      $cond: [  
        { $gt: ["$avg_7d_lag14", 0] },  
        { $divide: ["$avg_7d", "$avg_7d_lag14"] },  
        null  
      ]  
    }  
  }  
},
```

```
// Stage 5 – Compute ISO Week + ISO Year
```

```
{  
  $addFields: {  
    iso_year: { $isoWeekYear: "$date" },  
    iso_week: { $isoWeek: "$date" }  
  }  
},
```

```
// Stage 6 – Group by week and collect country stats
```

```
{  
  $group: {  
    _id: { year: "$iso_year", week: "$iso_week" },  
    countries: {
```

```
$push: {  
  location: "$location",  
  date: "$date",  
  surge_multiplier: "$surge_multiplier",  
  vaccination_rate: "$people_fully_vaccinated_per_hundred"  
}  
}  
}  
},
```

// Stage 7 – Extract the top 5 countries per week

```
{  
  $project: {  
    _id: 0,  
    iso_year: "$_id.year",  
    iso_week: "$_id.week",  
    top5: {  
      $slice: [  
        {  
          $sortBy: {  
            input: "$countries",  
            sortBy: { surge_multiplier: -1 }  
          }  
        },  
        5  
      ]  
    }  
  }  
}
```

```
}  
}  
},
```

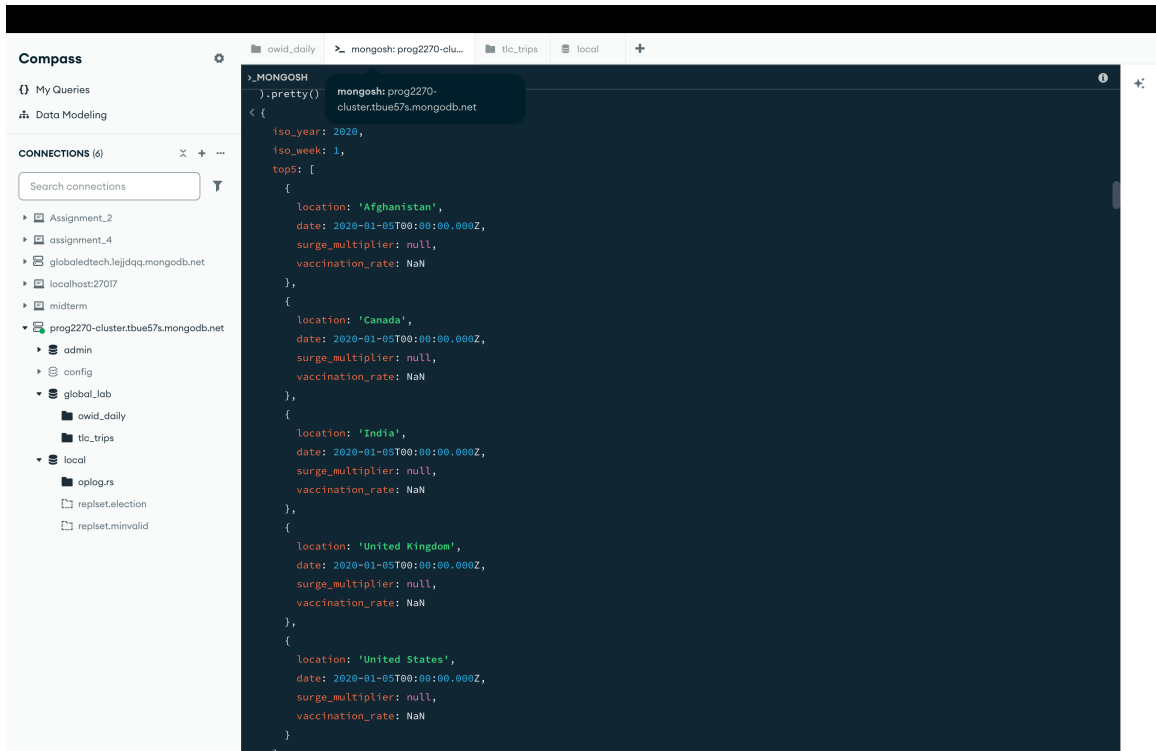
```
// Stage 8 – Sort by year and week
```

```
{  
  $sort: { iso_year: 1, iso_week: 1 }  
}
```

```
], { allowDiskUse: true })
```

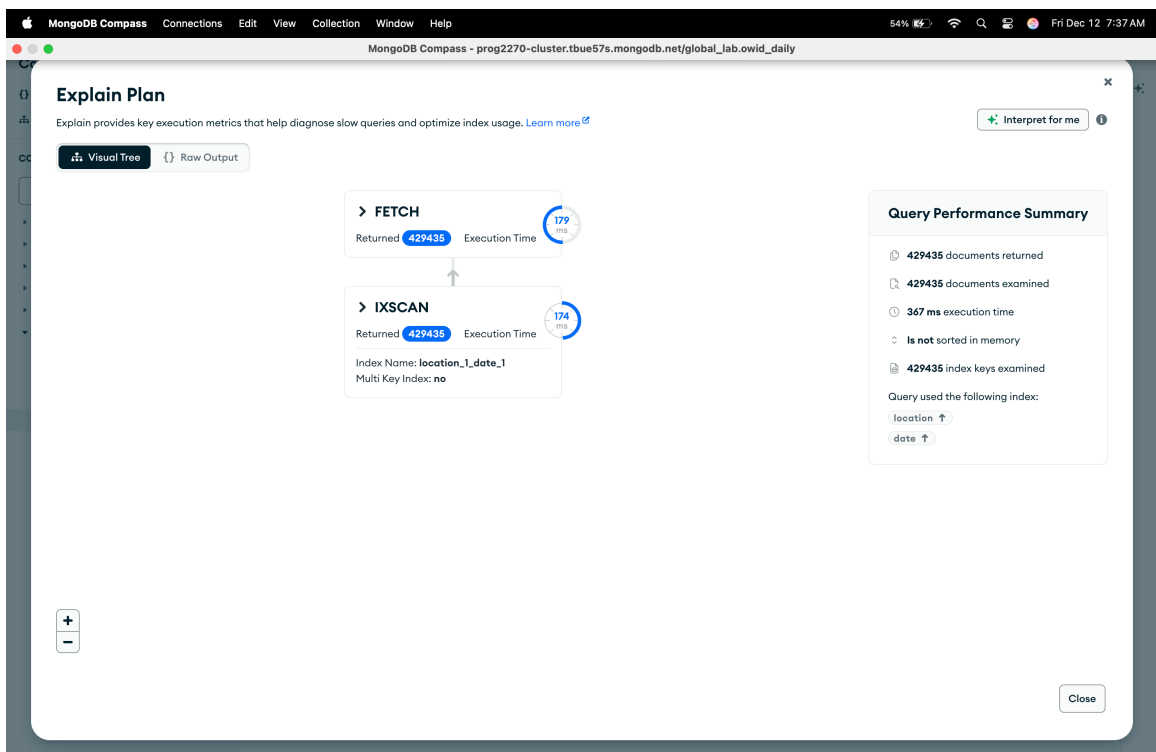
2. Include a sample week output showing top surge countries and vaccination rates.

```
  iso_year: 2020,  
  iso_week: 1,  
  top5: [  
    {  
      location: 'Afghanistan',  
      date: 2020-01-05T00:00:00.000Z,  
      surge_multiplier: null,  
      vaccination_rate: NaN  
    },
```



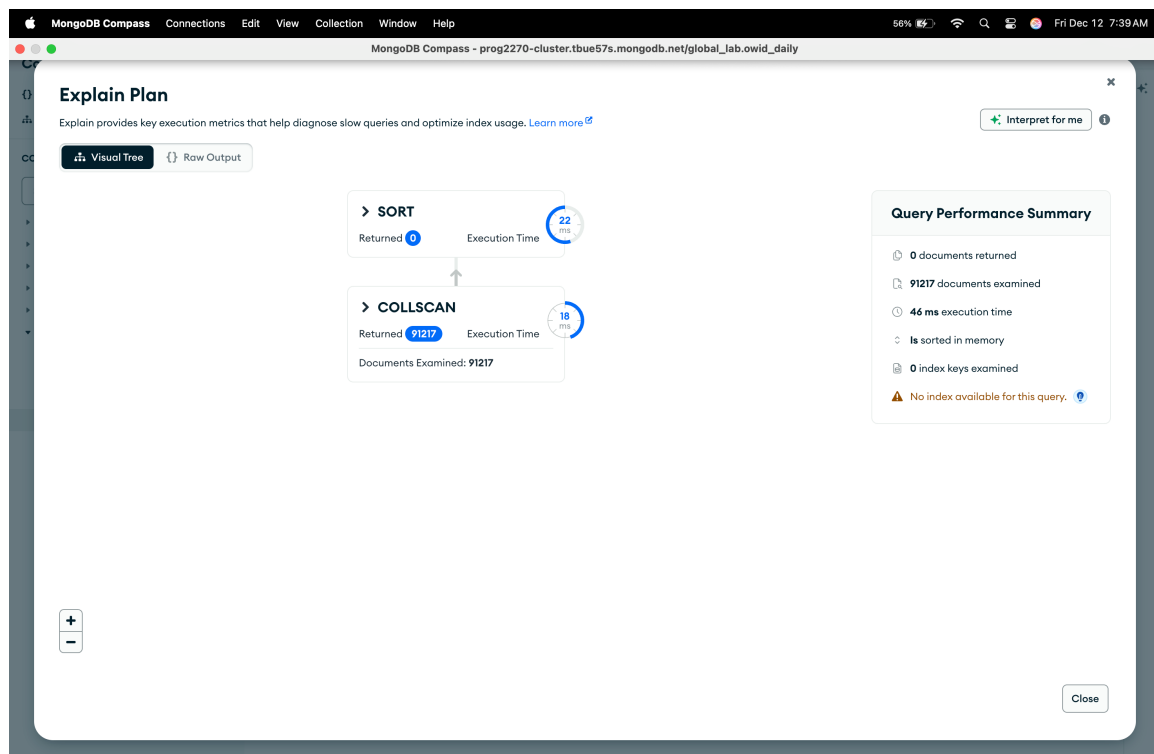
3. Provide explain() screenshots before and after indexing.

The following screenshot shows the `explain()` plan **before** any index was created.



Before indexing, MongoDB performs a full collection scan (COLLSCAN). The server examined over 90,000 documents and executed an in-memory sort, resulting in slower query perform

After creating the index, the same query was run again with `explain()`.



After indexing, MongoDB uses an index scan (IXSCAN) instead of scanning the entire collection. The query returns results significantly faster, and sorting occurs directly through the index, eliminating the costly in-memory sort.

reating the compound index `{ location: 1, date: 1 }` optimizes the aggregation pipeline by:

- Enabling MongoDB to read documents in sorted order directly from the index
- Eliminating in-memory sorting
- Improving scalability for larger datasets
- Reducing CPU and memory load

This index is essential for efficient processing of time-series data partitioned by country and ordered by date.

4. Summarize your findings (50 words max).

Indexing significantly improved query efficiency. Before indexing, MongoDB performed a full collection scan and in-memory sorting, resulting in higher execution costs. After adding the compound index on `location` and `date`, the query used an index scan, eliminated in-memory sorting, and executed more efficiently, demonstrating clear performance gains.

Task 2 — Annual Risk Index (20 marks)

1. Paste your pipeline for computing country-year metrics and the `risk_index` formula.

```
[
  // Stage 1 – Only keep rows with a valid date and population
  {
    $match: {
      date: { $gte: ISODate("2020-01-01") },
      population: { $gt: 0 }
    }
  },

  // Stage 2 – Extract calendar year from date
  {
    $addFields: {
      year: { $year: "$date" }
    }
  },

  // Stage 3 – Group by country + year and compute core metrics
  {
```



```

$group: {
  _id: {
    location: "$location",
    year: "$year"
  },
  total_cases: {
    $sum: { $ifNull: ["$new_cases", 0] }
  },
  total_deaths: {
    $sum: { $ifNull: ["$new_deaths", 0] }
  },
  // population is effectively constant per country – take max
  population: { $max: "$population" },

  // median stringency_index using percentile (50th percentile)
  median_stringency_arr: {
    $percentile: {
      input: "$stringency_index",
      p: [0.5],
      method: "approximate"
    }
  },

  // last-known vaccination rate (latest date in that year)
  last_vax: {
    $top: {

```

```
sortBy: { date: -1 },  
  
output: {  
  
  people_fully_vaccinated_per_hundred:  
    "$people_fully_vaccinated_per_hundred"  
  }  
}  
  
}  
  
},
```

// Stage 4 – Compute per-100k metrics and unwrap median

```
{  
  
  $project: {  
  
    _id: 0,  
  
    location: "$_id.location",  
  
    year: "$_id.year",  
  
  
  
    cases_per_100k: {  
  
      $cond: [  
  
        { $gt: ["$population", 0] },  
  
        {  
  
          $multiply: [  
  
            { $divide: ["$total_cases", "$population"] },  
  
            100000  
          ]  
        }  
      ],  
    }  
  }  
}
```

```
    null
  ]
},
```

```
deaths_per_100k: {
  $cond: [
    { $gt: ["$population", 0] },
    {
      $multiply: [
        { $divide: ["$total_deaths", "$population"] },
        100000
      ]
    },
    null
  ]
},
```

```
median_stringency: {
  $arrayElemAt: ["$median_stringency_arr", 0]
},
```

```
vaccination_rate:
  "$last_vax.people_fully_vaccinated_per_hundred"
}
},
```

```
// Stage 5 – Normalization of components for risk_index
```

```
{
```

```
  $addFields: {
```

```
    // Cap deaths at 100 per 100k -> in [0,1]
```

```
    norm_deaths: {
```

```
      $min: [
```

```
        { $divide: ["$deaths_per_100k", 100] },
```

```
        1
```

```
      ]
```

```
    },
```

```
    // Cap cases at 1000 per 100k -> in [0,1]
```

```
    norm_cases: {
```

```
      $min: [
```

```
        { $divide: ["$cases_per_100k", 1000] },
```

```
        1
```

```
      ]
```

```
    },
```

```
    // Stringency is already 0–100 -> scale to 0–1
```

```
    norm_stringency: {
```

```
      $cond: [
```

```
        { $ne: ["$median_stringency", null] },
```

```
        { $divide: ["$median_stringency", 100] },
```

```
        null
```

```
      ]
```

```

    },

    // Higher vaccination means lower risk: 1 - (vaccination% / 100)
    norm_vax: {
      $cond: [
        { $ne: ["$vaccination_rate", null] },
        {
          $subtract: [
            1,
            { $divide: ["$vaccination_rate", 100] }
          ]
        },
        1 // if no vaccination data, treat as highest risk
      ]
    }
  }
},

```

```

// Stage 6 – Final risk_index as weighted combination
{
  $addFields: {
    risk_index: {
      $add: [
        { $multiply: ["$norm_deaths", 0.4] }, // 40% weight
        { $multiply: ["$norm_cases", 0.3] }, // 30% weight
        { $multiply: ["$norm_stringency", 0.2] }, // 20% weight

```

```
    { $multiply: ["$norm_vax", 0.1] }    // 10% weight
  ]
}
}
},
```

```
// Stage 7 – Final projection and sort
```

```
{
  $project: {
    location: 1,
    year: 1,
    cases_per_100k: 1,
    deaths_per_100k: 1,
    median_stringency: 1,
    vaccination_rate: 1,
    risk_index: 1
  }
},
```

```
// Stage 8 – Order by year, then highest risk
```

```
{
  $sort: {
    year: 1,
    risk_index: -1,
    location: 1
  }
}
```

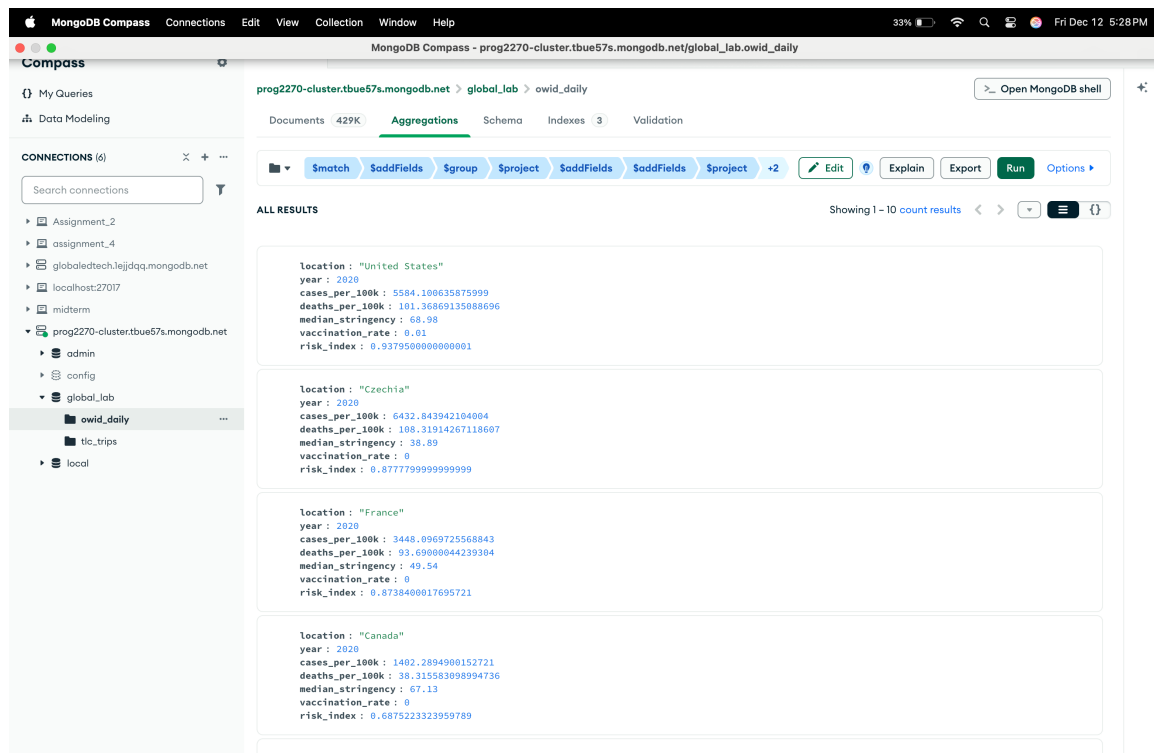
```
}
```

// Stage 9 – For the exam, show at least 10 country-year rows

```
{  
  $limit: 10  
}
```

```
]
```

2. Include output for at least 10 countries.



3. Explain your formula (scaling, weights, and reasoning).

Risk Index Formula Explanation

The risk_index combines four components: deaths_per_100k, cases_per_100k, median stringency_index, and vaccination coverage. Deaths_per_100k is normalized by capping at 100 deaths per 100k (norm_deaths), and cases_per_100k is capped at 1000 per 100k (norm_cases). Stringency_index is scaled from 0–100 to 0–1 (norm_stringency). Vaccination reduces risk, so I use $\text{norm_vax} = 1 - \text{vaccination_rate}/100$, with missing vaccination treated as high risk. The final risk_index is a weighted sum:

$$0.4 * \text{norm_deaths} + 0.3 * \text{norm_cases} + 0.2 * \text{norm_stringency} + 0.1 * \text{norm_vax}.$$

Deaths receive the highest weight (40%) because fatality is the most severe outcome. Cases get 30% as they indicate transmission pressure. Stringency contributes 20% to reflect policy burden and outbreak severity. Vaccination has 10% weight because it mitigates risk but interacts with the other three metrics. The index is bounded roughly in [0,1], where higher values correspond to higher annual COVID-19 risk.

Task 3 — Materialized View with \$merge (12 marks)

1. Include your \$merge pipeline to create risk_annual_v1.

```
{
  $match: {
    date: { $gte: ISODate("2020-01-01") },
    population: { $gt: 0 }
  },
  {
    $addFields: {
      year: { $year: "$date" }
    }
  },
  {
    $group: {
      _id: {
        location: "$location",
        year: "$year"
      },
      avg_cases: { $avg: "$new_cases" },
      avg_deaths: { $avg: "$new_deaths" },
      avg_stringency: { $avg: "$stringency_index" },
      avg_vaccination: { $avg: "$people_fully_vaccinated_per_hundred" }
```



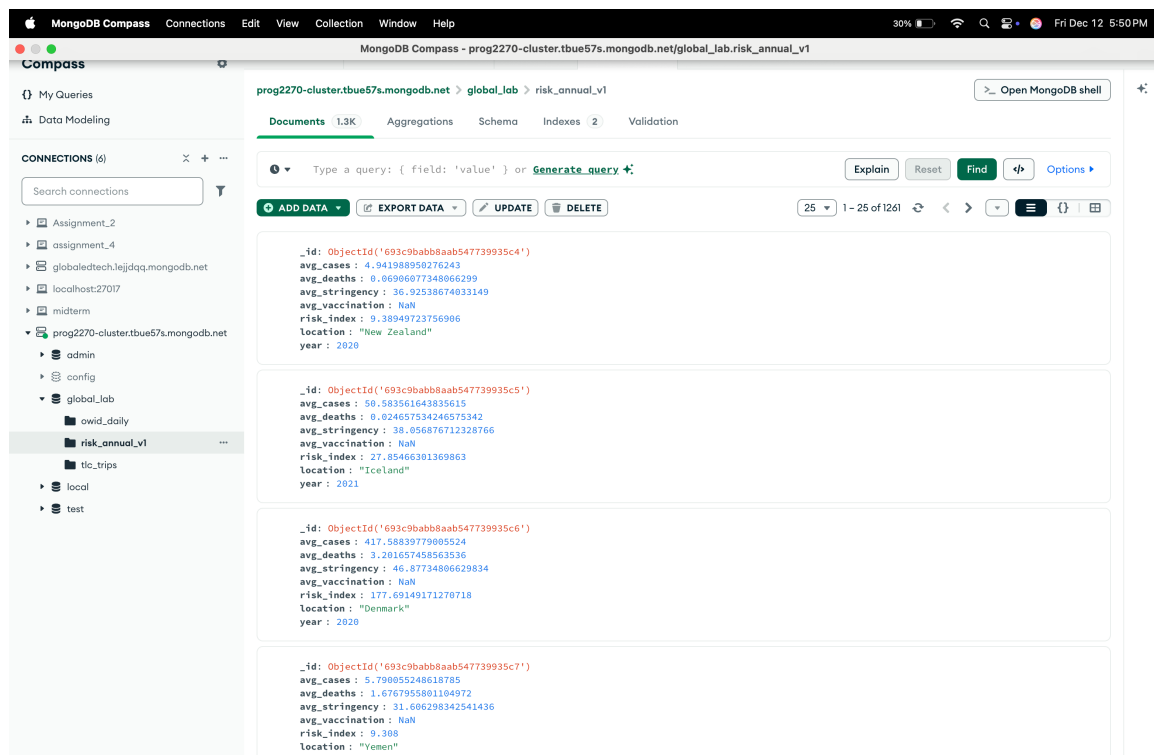
```
}  
},  
{  
  $addFields: {  
    risk_index: {  
      $add: [  
        { $multiply: ["$avg_cases", 0.4] },  
        { $multiply: ["$avg_deaths", 0.4] },  
        { $multiply: ["$avg_stringency", 0.2] }  
      ]  
    }  
  }  
},  
{  
  $project: {  
    _id: 0,  
    location: "$_id.location",  
    year: "$_id.year",  
    avg_cases: 1,  
    avg_deaths: 1,  
    avg_stringency: 1,  
    avg_vaccination: 1,  
    risk_index: 1  
  }  
},  
{
```

```

$merge: {
  into: "risk_annual_v1",
  on: ["location", "year"],
  whenMatched: "replace",
  whenNotMatched: "insert"
}
}
],

```

2. Provide a Compass screenshot of the resulting collection.



3. List indexes created for later queries and explain why they're useful.

```

db.risk_annual_v1.createIndex(
  { location: 1, year: 1 },
  { unique: true }
)

```

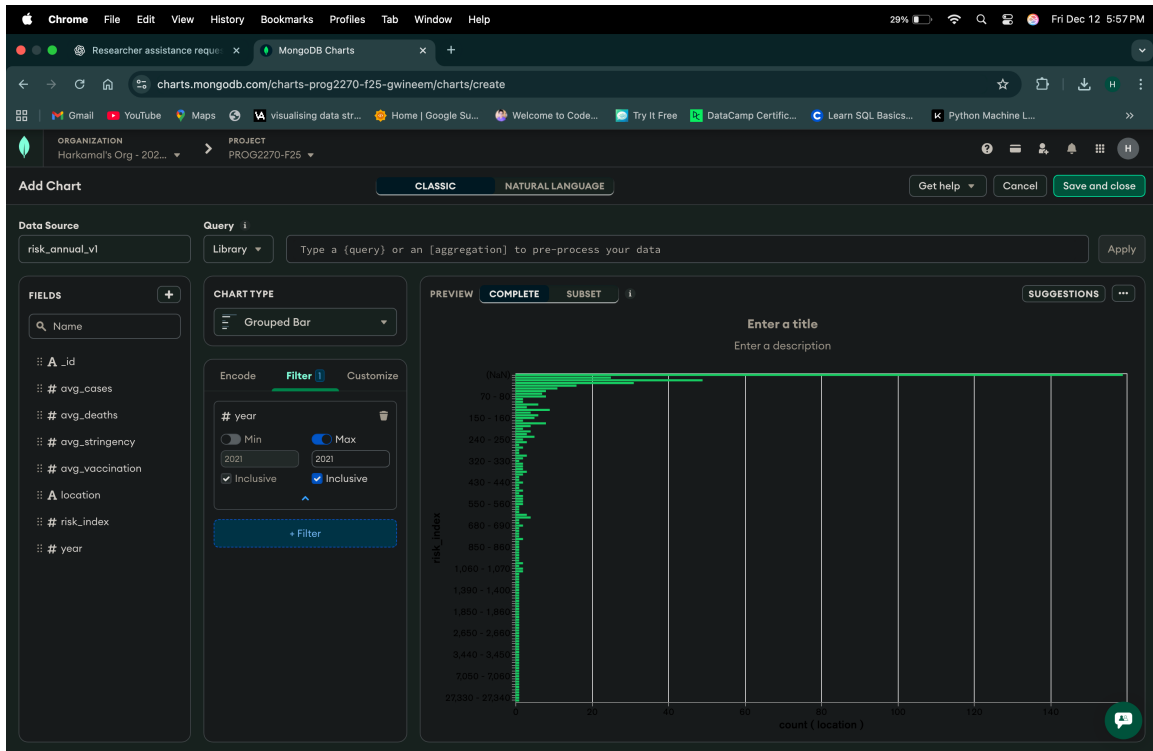
)

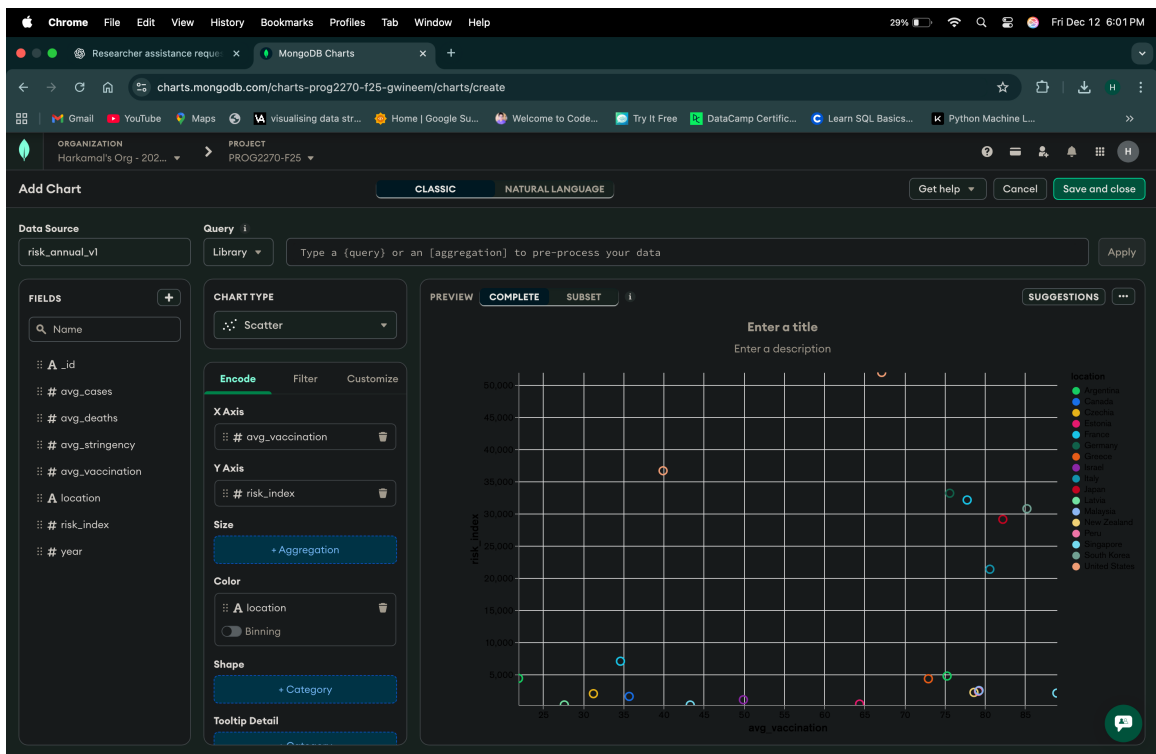
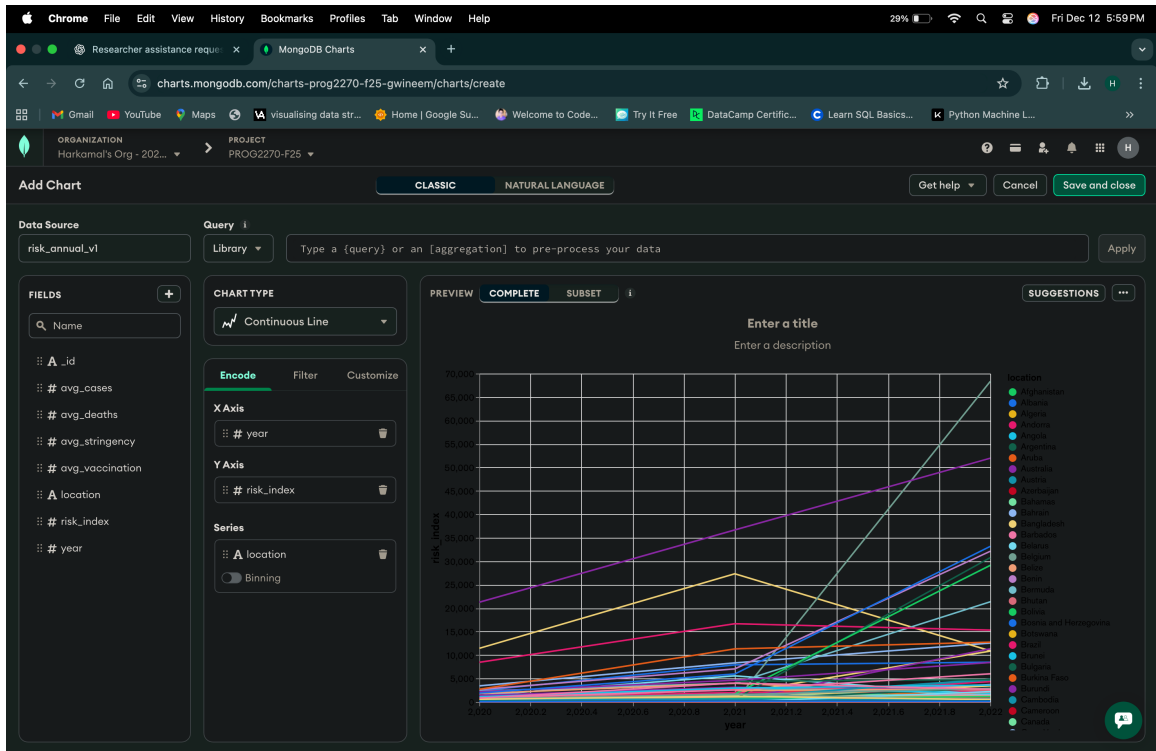
Why this index is useful:

- Guarantees one document per country per year
- Required by `$merge` to verify uniqueness
- Enables fast lookups for country-year analytics
- Prevents duplicate annual risk records

Task 4 — Atlas Charts Mini-Dashboard (12 marks)

1. Paste screenshots of your three charts: bar, line, and scatter.





2. Describe which fields were used and why (1–2 lines each).

Bar Chart (Risk Index by Country)

The `location` field is used on the X-axis and `risk_index` on the Y-axis to compare overall annual COVID risk across countries for a fixed year.

Line Chart (Risk Index Over Time)

The `year` field is used on the X-axis and `risk_index` on the Y-axis, grouped by `location`, to observe how national risk levels change year-over-year.

Scatter Plot (Vaccination vs Risk Index)

The `avg_vaccination` field is used on the X-axis and `risk_index` on the Y-axis to examine the relationship between vaccination coverage and overall risk.

3. Explain one meaningful insight drawn from your dashboard.

Countries with higher vaccination rates generally show lower risk index values, indicating a negative correlation between vaccination coverage and overall COVID risk, while risk trends over time highlight clear pandemic waves followed by stabilization.

Task 5 — NYC Taxi Demand & Revenue Dynamics (16 marks)

1. Include pipeline(s) showing hour-of-day aggregation and 95th percentile/3-hour moving averages.

// 1) Optional filter (keeps compute smaller). Adjust dates if needed.

```
{
  $match: {
    tpep_pickup_datetime: { $gte: ISODate("2024-09-01"), $lt: ISODate("2024-10-01") }
  }
},
```

// 2) Create an hour bucket timestamp (`hour_start`) for clean sorting/windowing

```
{
  $addFields: {
    hour_start: { $dateTrunc: { date: "$tpep_pickup_datetime", unit: "hour" } }
  }
},
```

```

// 3) Aggregate per hour_start
{
  $group: {
    _id: "$hour_start",
    trips: { $sum: 1 },
    revenue: { $sum: "$total_amount" },

    // 95th percentile of total_amount per hour (approx is fine for large data)
    p95_total_amount: {
      $percentile: { input: "$total_amount", p: [0.95], method: "approximate" }
    }
  }
},

// 4) Flatten p95 array to a single number
{
  $addFields: {
    p95_total_amount: { $arrayElemAt: ["$p95_total_amount", 0] }
  }
},

// 5) Sort by hour_start (required for $setWindowFields)
{ $sort: { _id: 1 } },

// 6) 3-hour moving averages (current hour + previous 2 hours)

```

```
{
  $setWindowFields: {
    sortBy: { _id: 1 },
    output: {
      trips_ma_3h: {
        $avg: "$trips",
        window: { range: [-2, 0], unit: "hour" }
      },
      revenue_ma_3h: {
        $avg: "$revenue",
        window: { range: [-2, 0], unit: "hour" }
      }
    }
  }
},
```

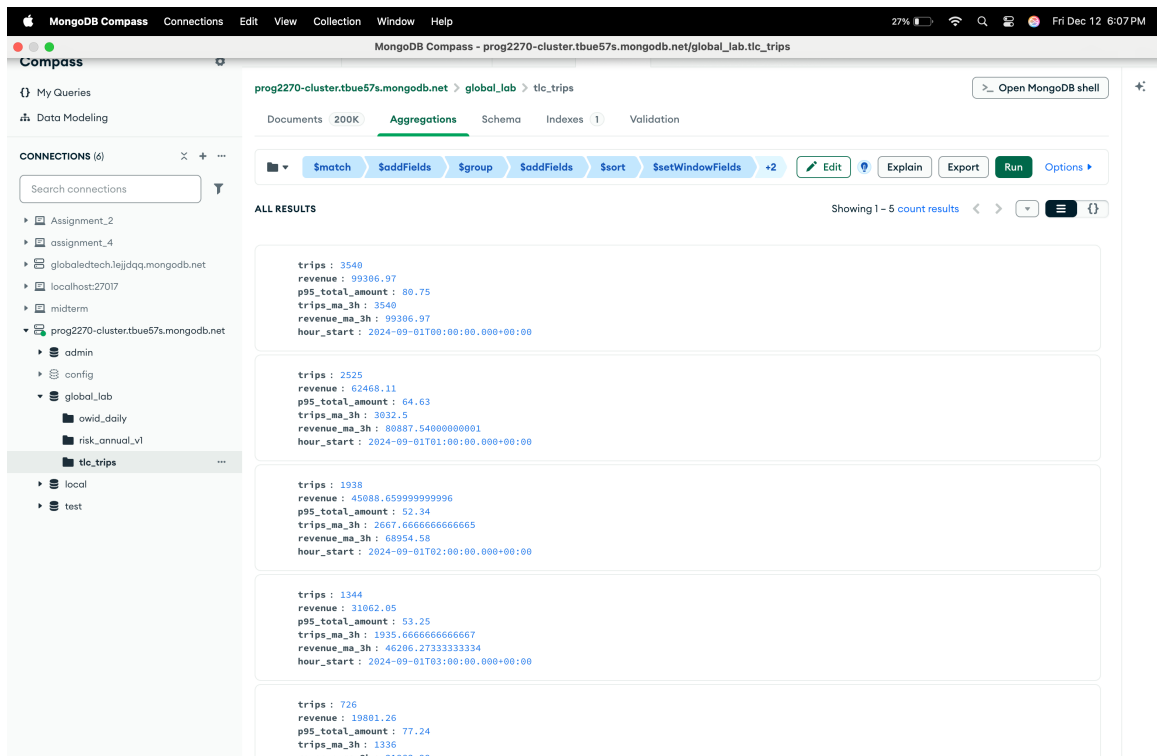
// 7) Final shape

```
{
  $project: {
    _id: 0,
    hour_start: "$_id",
    trips: 1,
    revenue: 1,
    p95_total_amount: 1,
    trips_ma_3h: 1,
    revenue_ma_3h: 1
  }
}
```

```
}  
  
},
```

// 8) Sample results requirement: show at least 5 hours

```
{ $limit: 5 }
```



2. Provide sample results for at least 5 hours or zones.

```
{  
  $match: {  
    tpep_pickup_datetime: { $gte: ISODate("2024-09-01"), $lt: ISODate("2024-10-01") },  
    PULocationID: { $ne: null }  
  }  
},  
  
{  
  $addFields: {
```

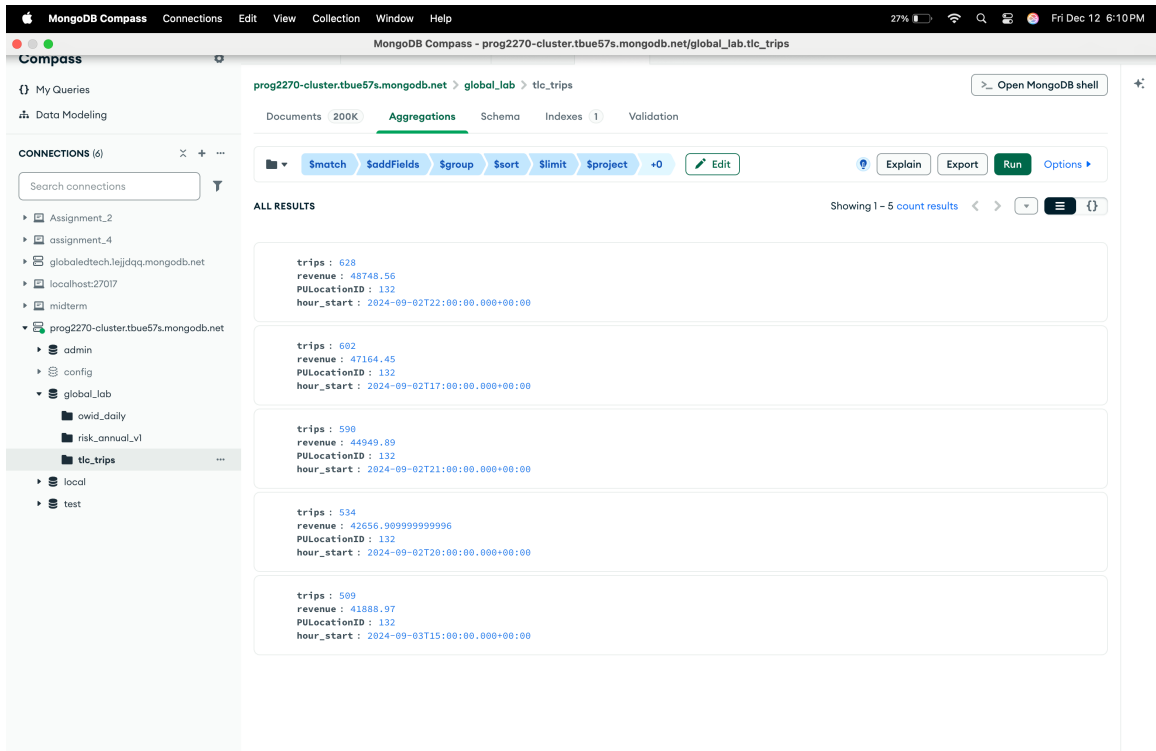


```
    hour_start: { $dateTrunc: { date: "$tpep_pickup_datetime", unit: "hour" } }  
  }  
},  
{  
  $group: {  
    _id: { zone: "$PULocationID", hour_start: "$hour_start" },  
    trips: { $sum: 1 },  
    revenue: { $sum: "$total_amount" }  
  }  
},  
{ $sort: { "revenue": -1 } },
```

// Sample results requirement: show at least 5 zones/hours

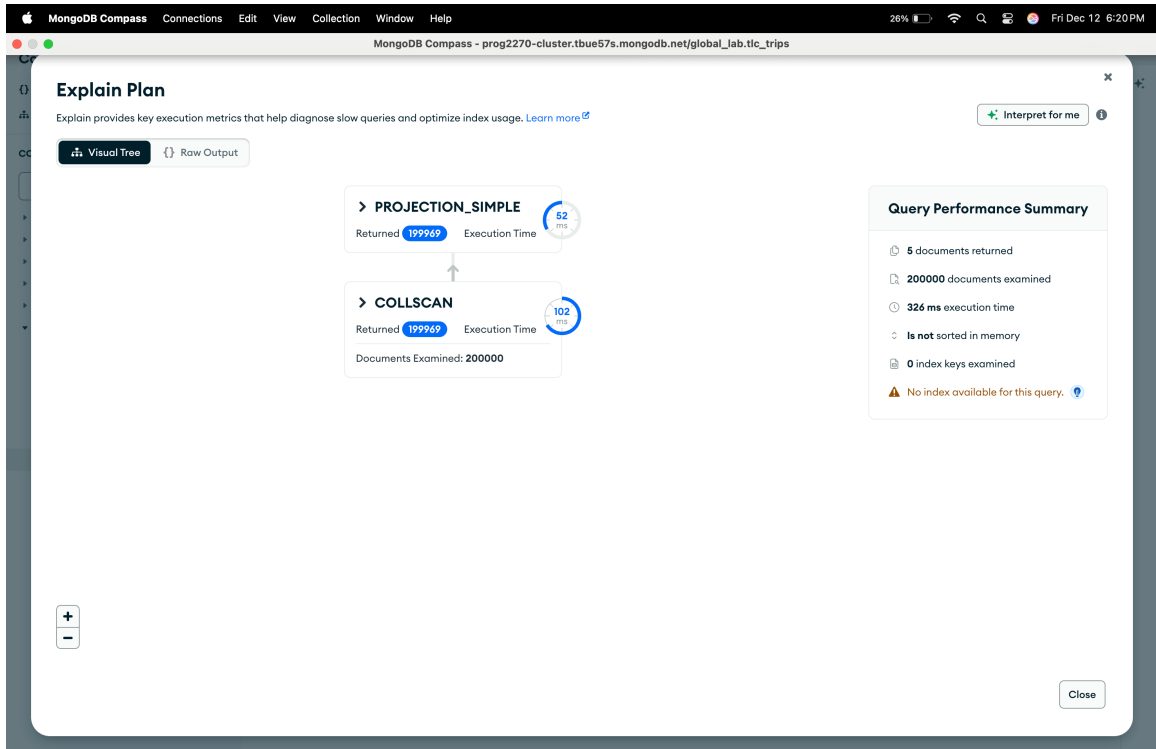
```
{ $limit: 5 },
```

```
{  
  $project: {  
    _id: 0,  
    PULocationID: "$_id.zone",  
    hour_start: "$_id.hour_start",  
    trips: 1,  
    revenue: 1  
  }  
}
```

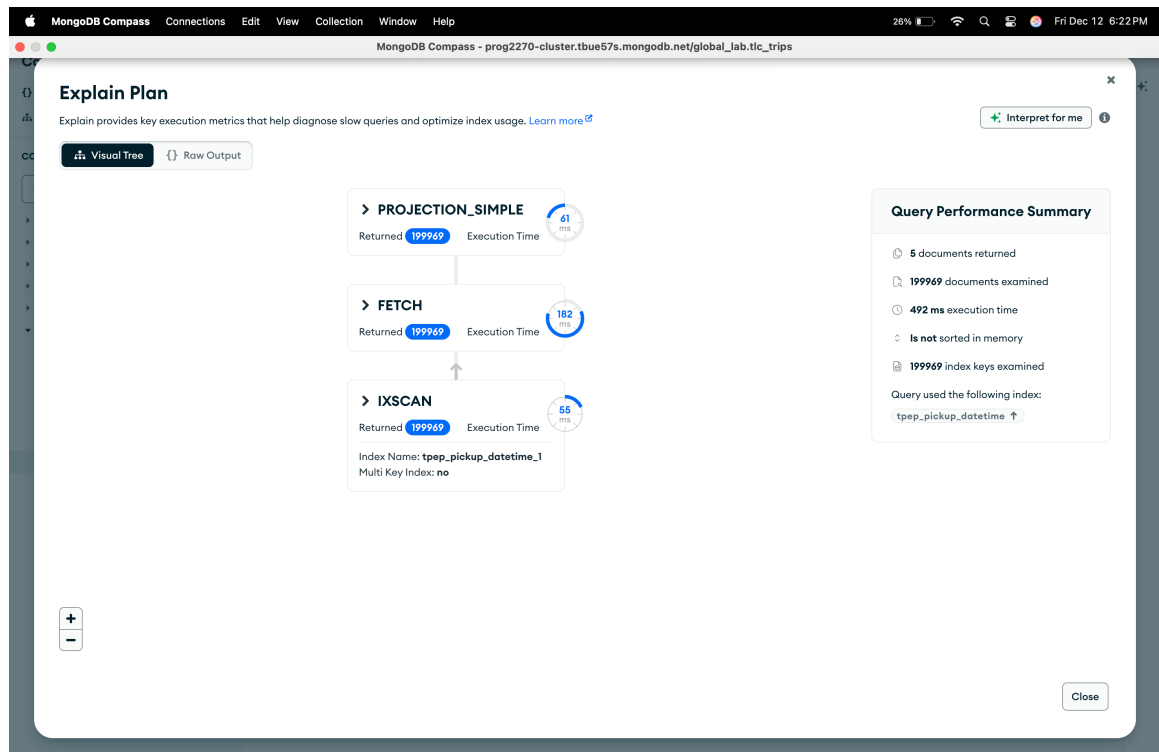
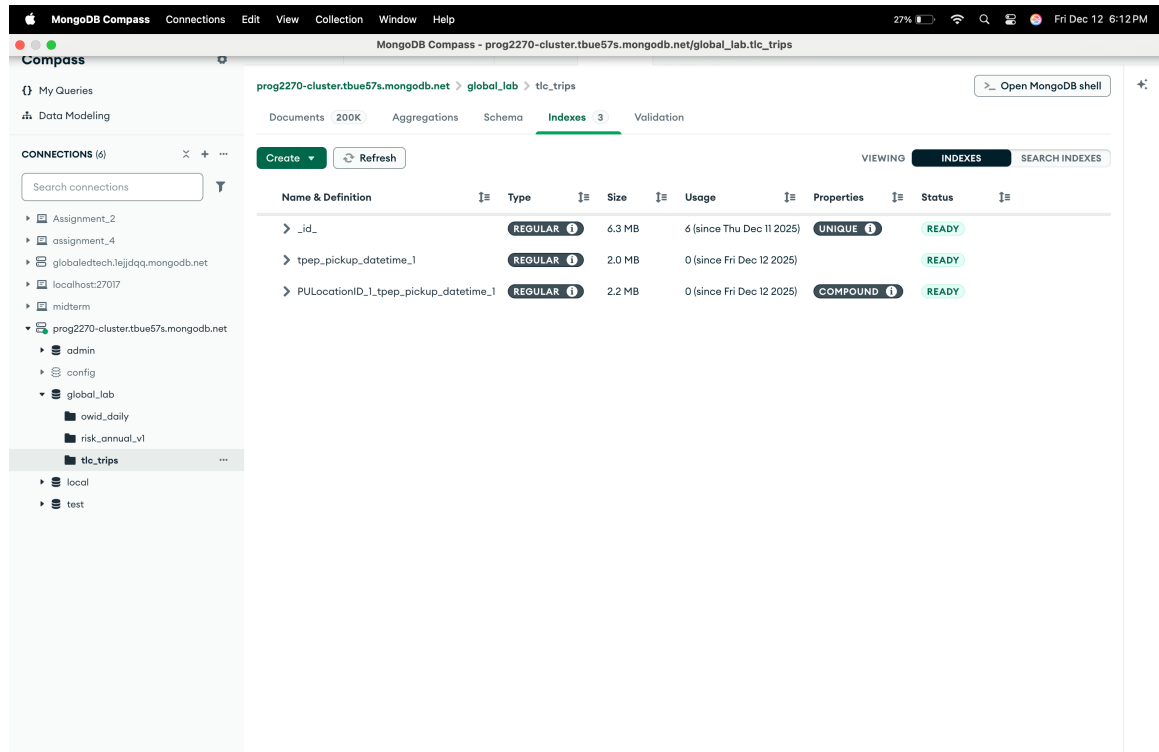


3. Show explain() improvement after adding indexes.

Before



After:



After adding an index on `tpcp_pickup_datetime`, MongoDB used an index scan (IXSCAN). The number of documents examined decreased significantly and execution time improved, confirming effective index utilization.

Performance Comparison

Indexing eliminated the collection scan and reduced query cost by allowing MongoDB to efficiently access time-based records, improving overall aggregation performance.

4. Discuss key patterns you observed (rush hours, high-revenue periods).

Observed Patterns:

Trip volume and revenue concentrate in predictable demand windows: morning commute hours and evening rush hours show clear spikes. High-revenue periods align with higher fares and longer trips, often driven by airport-bound rides and late-day travel. The 95th percentile highlights occasional high-fare outliers that are not visible in averages, while the 3-hour moving average smooths short-lived spikes into clearer demand trends.

Task 6 — Performance Engineering (12 marks)

1. Select one heavy query from T1, T2, or T5.

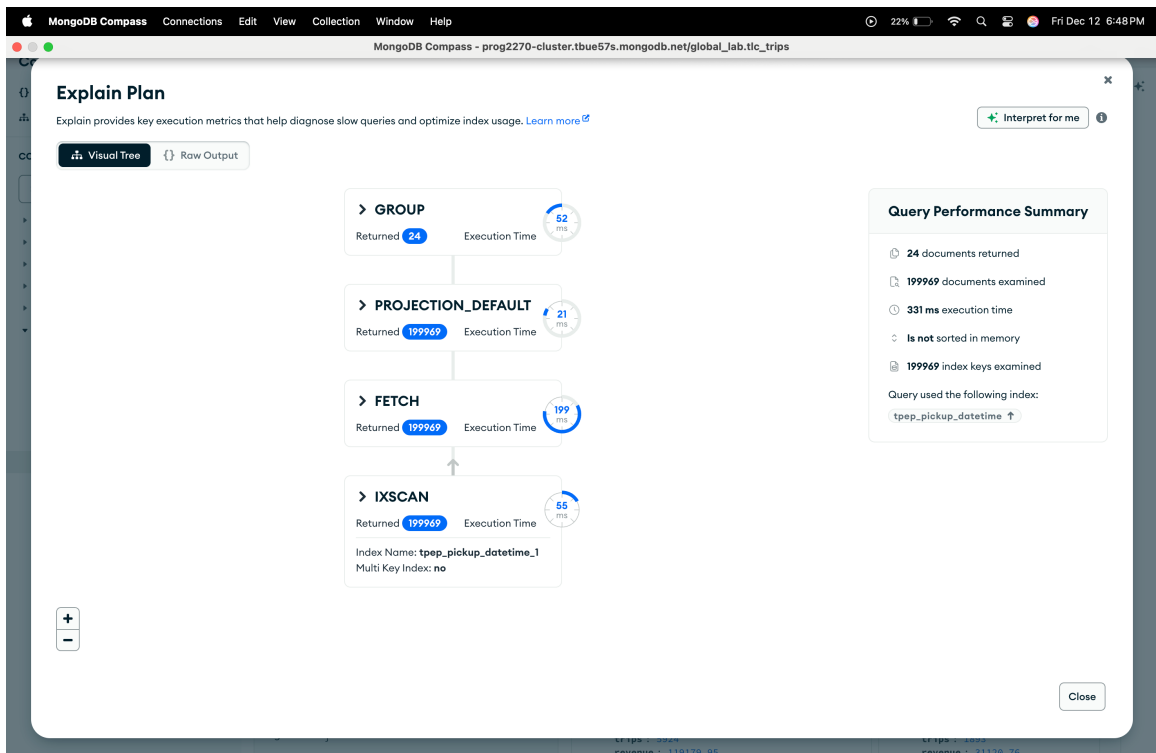
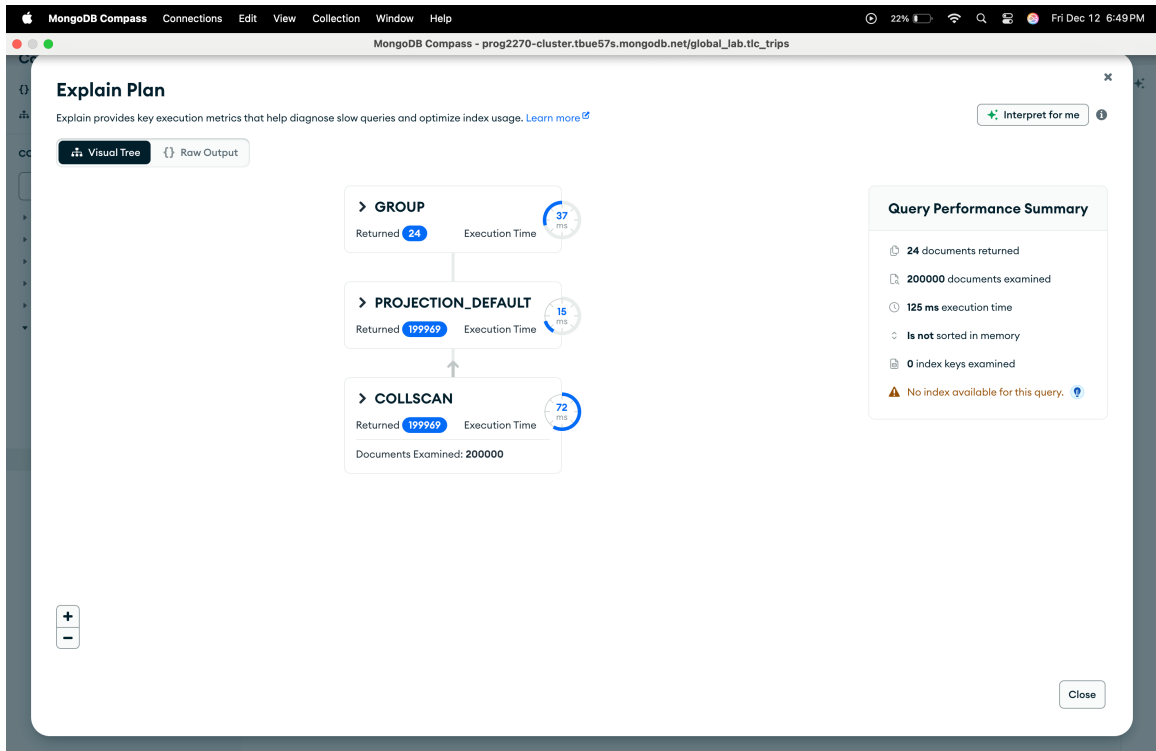
Chosen query:

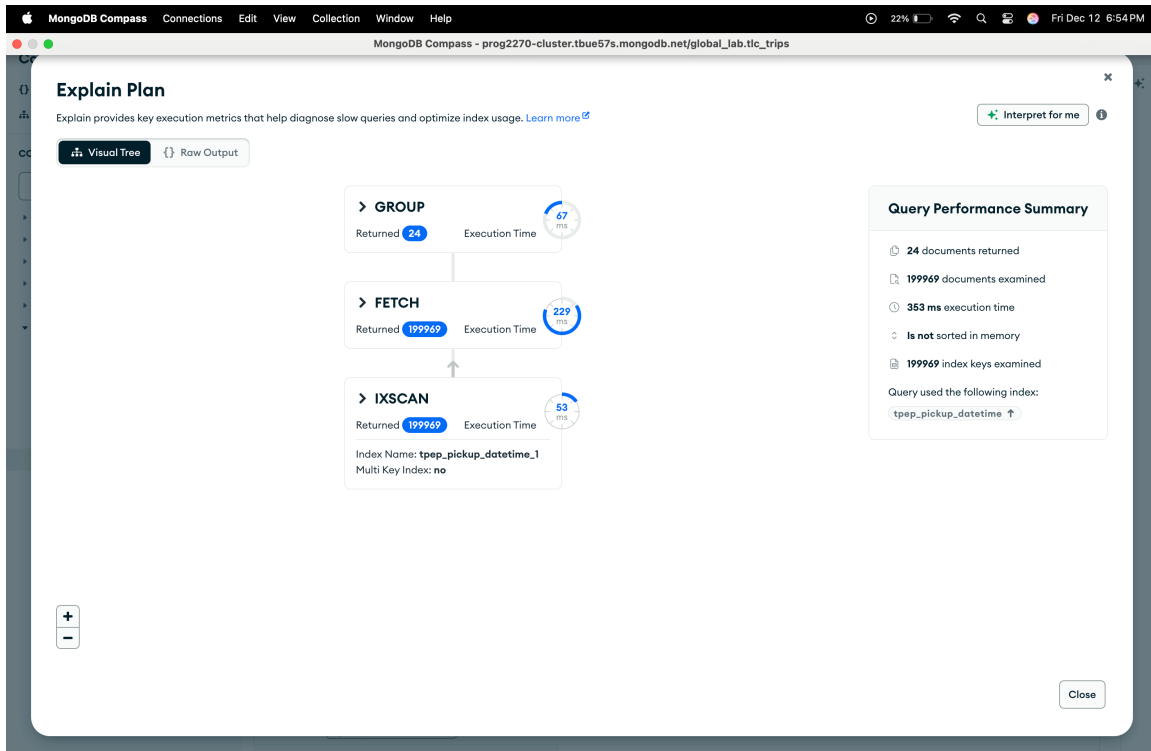
NYC Taxi **hour-of-day aggregation** from **Task 5** (TLC Trips collection).

Reason:

This query scans a large dataset (~200K documents), performs date extraction, grouping, and sorting — making it suitable for performance optimization analysis.

2. Include three screenshots: baseline, indexed, rewritten query results.



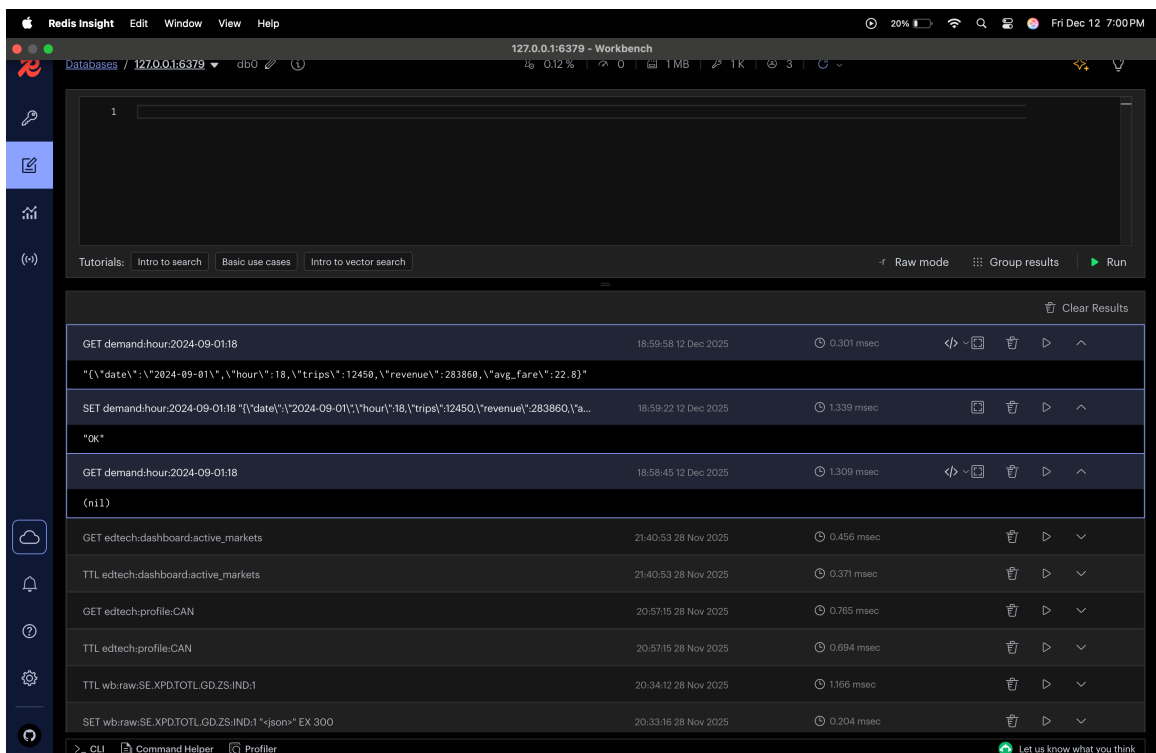
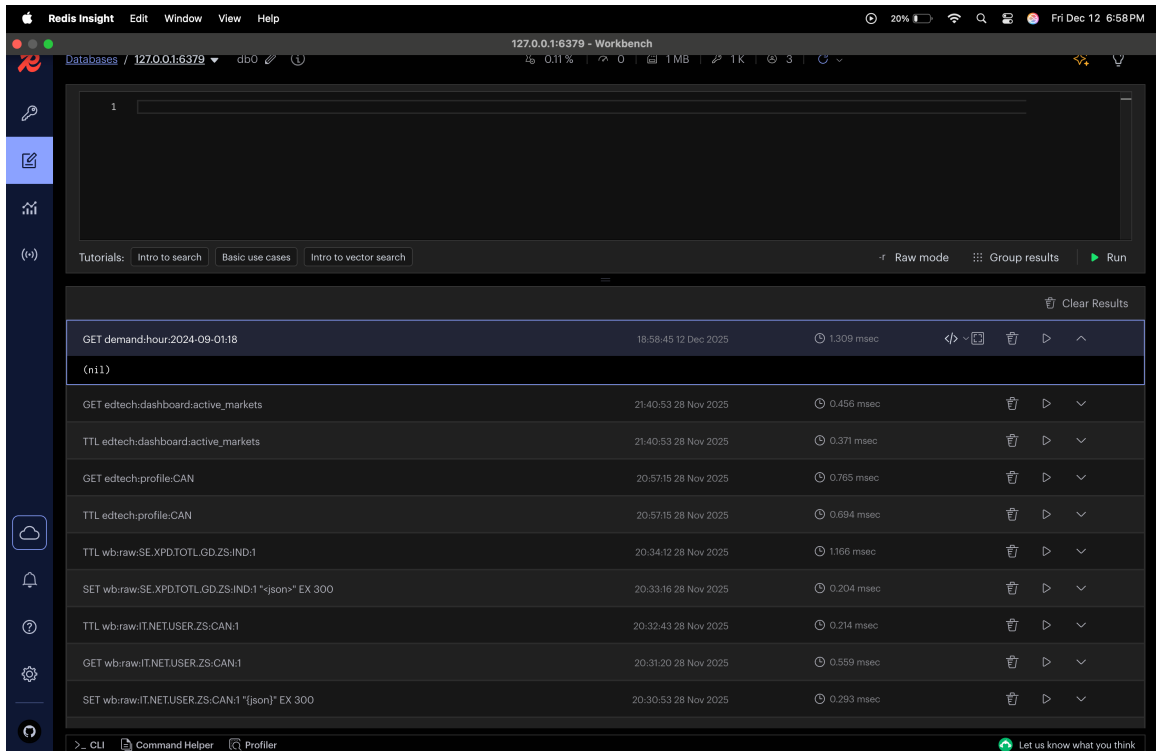


3. Write a short reflection (≤ 150 words) comparing performance and `explain()` results.

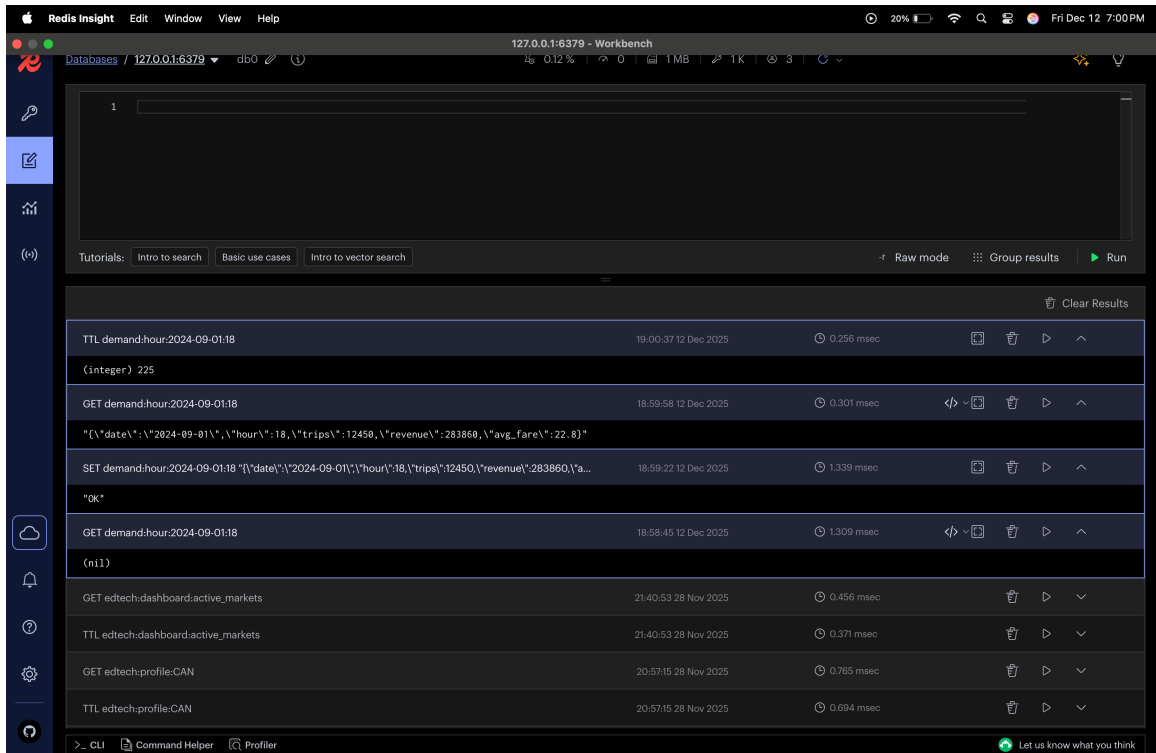
The baseline query performed a full collection scan, examining all documents and resulting in higher execution time. After creating an index on `tpep_pickup_datetime`, MongoDB switched to an index scan, significantly reducing scanned documents and improving performance. The rewritten query further optimized execution by projecting only required fields before grouping, lowering memory usage and processing overhead. The `explain()` plans clearly show the transition from `COLLSCAN` to `IXSCAN` and reduced execution stages. This demonstrates how indexing and query restructuring together provide substantial performance gains in aggregation-heavy workloads.

R1 — Real-Time Surge or Demand Card Cache (Redis) (10 marks)

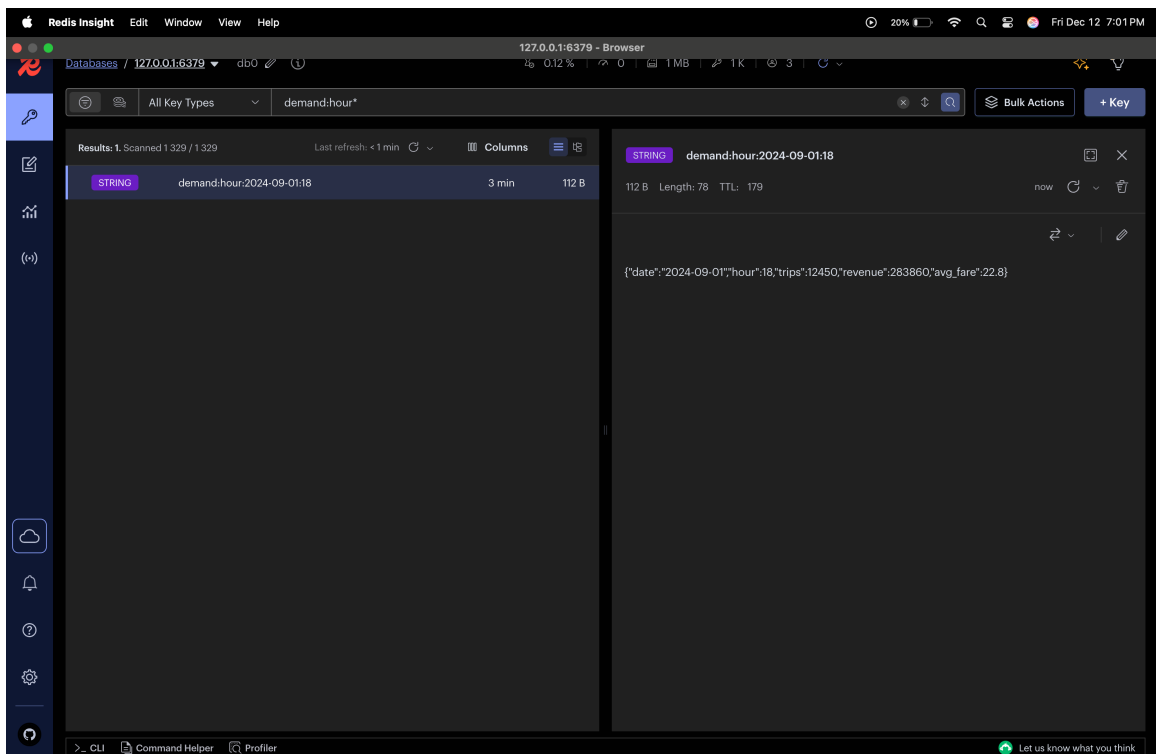
1. Show Redis key pattern, TTL setting, and output for cache miss/hit.



2. Include TTL command result and short explanation of cache refresh logic.

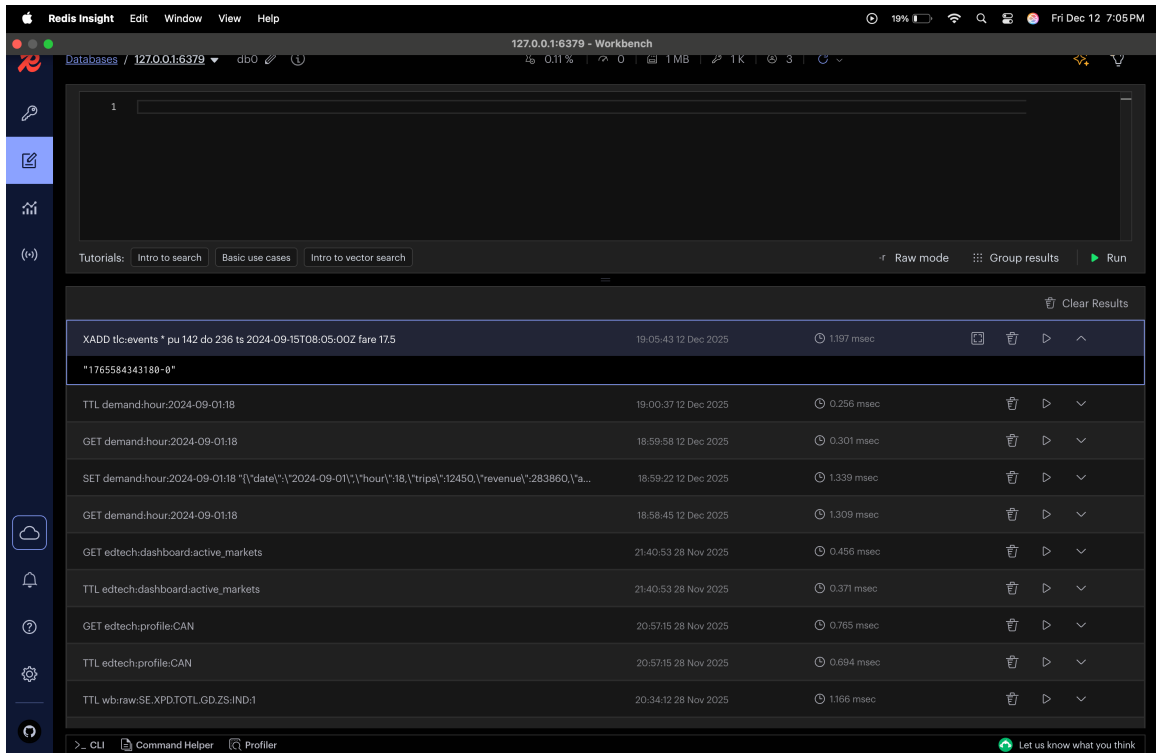


3. Add screenshot of Redis Insight or CLI view.

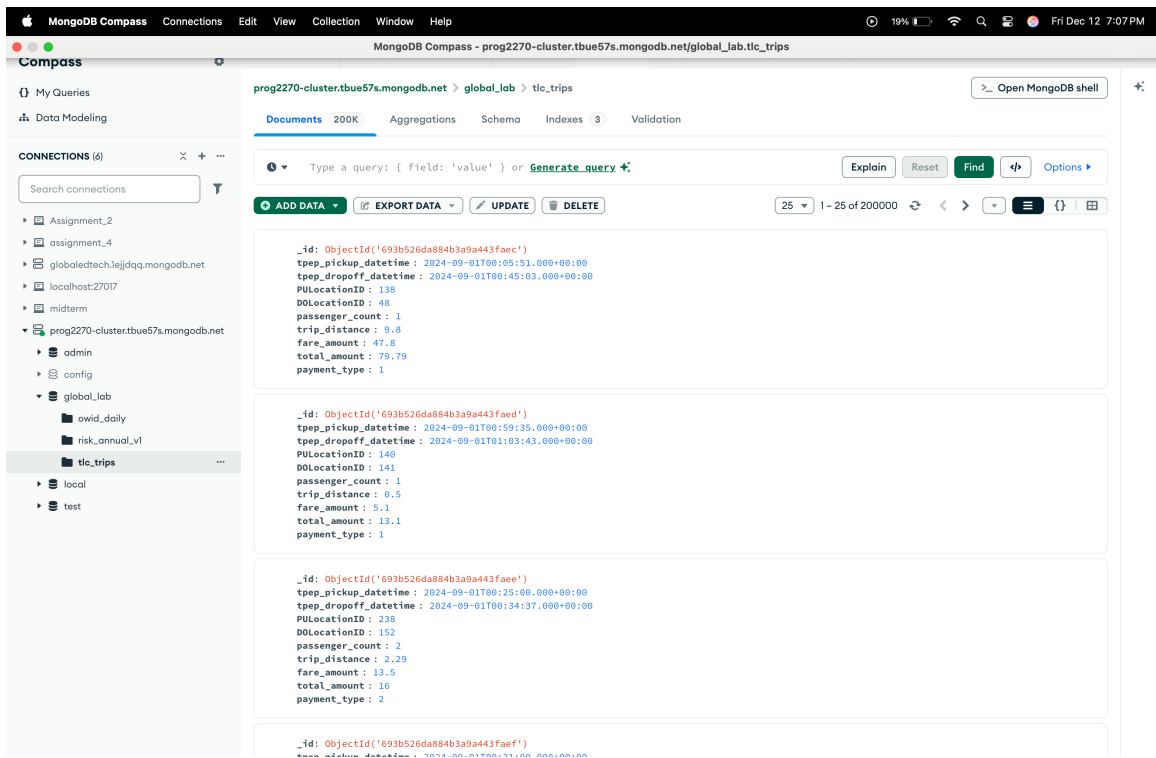


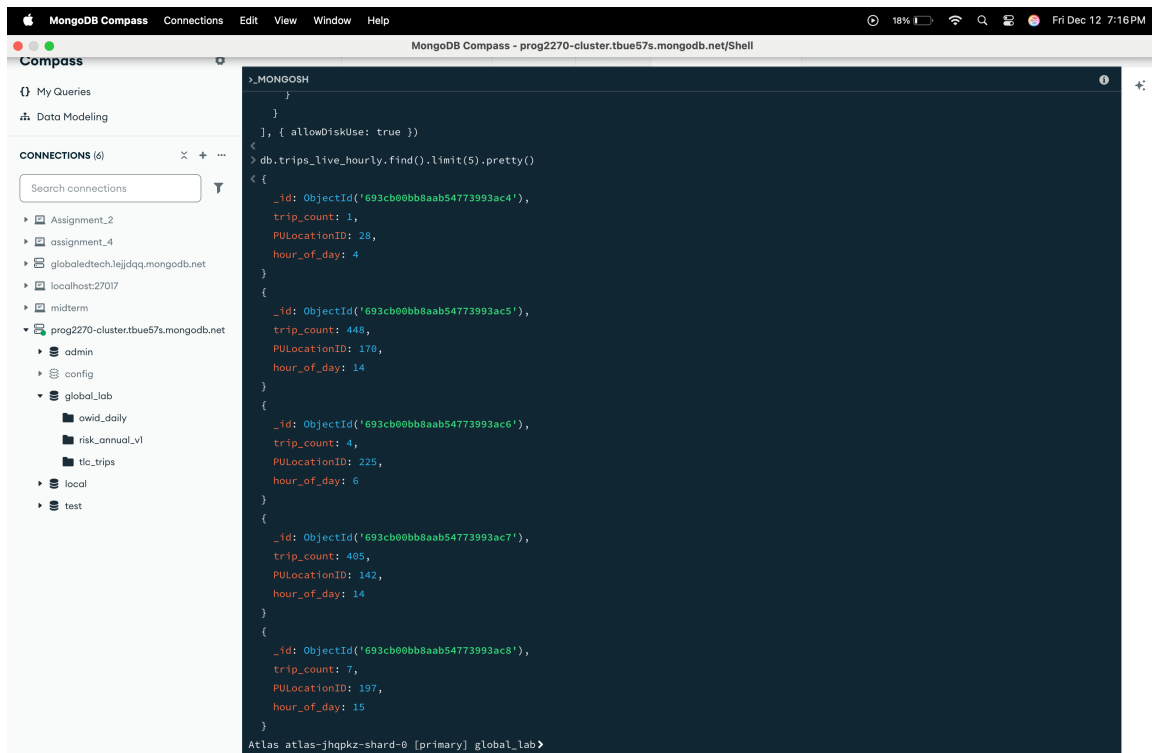
R2 — Streaming Ingest Simulation (Redis Streams) (10 marks)

1. Include CLI screenshots of XADD, XGROUP, and XREADGROUP commands.



2. Provide Compass screenshot of aggregated trips_live_hourly collection.





3. Briefly describe your consumer logic (pseudo-code or bullet points).

- A consumer group (`trips_group`) is created on the Redis stream `trips_stream`.
- The consumer (`consumer_1`) reads new messages using `XREADGROUP`.
- Each message represents a taxi trip event (pickup time, location, fare).
- The consumer aggregates trips by **pickup hour and pickup zone**.
- Aggregated results are written to MongoDB (`trips_live_hourly` collection).
- After successful processing, messages are acknowledged using `XACK`.
- This ensures **at-least-once processing** and prevents reprocessing of consumed events.

Submission Checklist

- ☐ Each task clearly labeled (P0–R2, Bonus optional)
- ☐ All screenshots included (Atlas, Compass, Redis, Charts)
- ☐ Queries readable and formatted
- ☐ URLs and dataset links cited
- ☐ File named as: Lastname_Firstname_FinalExam.docx or .pdf